

Learning what to keep under a real KV-cache budget

A learned cache policy, an 8 GiB GPU, and a strict memory budget. A completed pilot finds no consistent benefit from compression exposure—and an important evaluation confound.

Alireza Afshan September 22, 2026 Version 1.0

Alireza Afshan. September 22, 2026. Exploratory local pilot, with research and implementation assistance from Codex. Not peer reviewed.

Abstract

Can a small learned retention policy benefit from seeing compressed cache states during training? We compare paired lightweight policies trained on intact versus compression-exposed key/value features, with a frozen Qwen2.5-1.5B-Instruct model on one RTX 4060. The fixed experiment evaluates 33 conditions on the same 24 synthetic lookup and correction tasks, using physical eviction and BF16 or INT8 cache storage under 4 or 8 MiB resident budgets that include controller parameters. Exposure changes mean exact-format accuracy by -5.56, +4.17, +2.78, and 0 percentage points across the four settings, without a consistent benefit. At 8 MiB with INT8, both arms average 91.7% across three policy seeds, versus 100% for full cache. Resident cache-plus-controller storage falls from 25.33 to 7.99 MiB, while peak allocated GPU memory falls by only about 13.8 MiB. A post-primary diagnostic reveals an evaluation confound: recency scores 0/24 on the strict first-word endpoint but mentions the correct color in 16/24 longer answers. This study provides an inspectable implementation, a null exposure result, and a warning about conflating instruction retention with factual retrieval.

Question and scope

This is an independent, scaled KVP-style ranking experiment, inspired by Learning to Evict from Key-Value Cache. It is neither a reproduction of that paper's full benchmark nor a new claim to learned eviction. The question is whether fixed exposure to altered cache states helps a lightweight policy

operate under subsequent compression. The intervention is deliberately smaller than on-policy training.

Model weights remain BF16 and frozen. Only key/value storage and retention change. A KV-cache saving does not imply an equivalent saving in model-weight memory, nor establish that a larger model fits the hardware. This pilot does not test philosophy, mathematics, coding, long reasoning, multi-turn agents, or general language-model quality.

Prospective design

The primary protocol, implementation, configuration, and correctness tests were committed locally before the fixed run. A separate eight-task smoke gate required at least six correct full-cache answers; all eight were correct. The held-out primary run then completed once, with every condition retained. This is a locally frozen protocol, not an externally registered study.

The base model has 28 layers, 12 query heads, two KV heads per layer, and head dimension 128. We use local weights with recorded SHA-256 hashes, PyTorch 2.13.0+cu130, Transformers 4.57.6, and eager attention. The host GPU is an 8 GiB RTX 4060. There are 32 training traces and 24 held-out tasks, half ordinary lookups and half later corrections to an earlier value. The three policy initialization seeds are 41, 42, and 43; train, smoke, and test generators use disjoint fixed seeds. Test prompts contain roughly 900 tokens and 30 irrelevant records. Answers are one of eight colors.

Each of 56 layer/head policies is a width-32, two-layer MLP, taking post-RoPE keys, values, and two absolute-position features. Together they contain 465,976 parameters, occupying 1,863,904 bytes (1.78 MiB) in FP32. Training samples Plackett-Luce rankings through Gumbel perturbations and uses a leave-one-out REINFORCE estimator of future-attention ranking cost. Each arm receives exactly 160 AdamW updates at learning rate 0.0003, with eight sampled rankings per update. The final update is used, without checkpoint selection. Future attention is available only in training; inference uses current cache entries.

Native and exposed policies share examples, retained positions, teacher utility labels, initial parameters, and update schedules. Training contexts contain both the target fact and any correction. In the exposed arm, preamble positions 4 through 63 are evicted before processing the last 64 context tokens, and the cache uses actual INT8 storage. This fixed intervention changes states but is not generated by the learned policy's own decisions. It also removes an irrelevant preamble rather than a task-critical fact. That narrow exposure is a limitation of the tested hypothesis.

At inference, all compressed methods protect the first four and most recent 16 cache slots. Sink-plus-recency and seeded random retention are simple controls. The learned policies rank the remaining candidates. Prefill uses 64-token chunks, followed by greedy decoding. Eviction occurs after every

chunk and generated step, and retained positions stay absolute even when physical cache slots change. The primary output cap is 12 tokens.

What the memory budget means

The budgets include physically allocated retained K/V tensors, FP16 quantization scales, int64 position metadata, and policy parameters. They exclude frozen model weights and temporary tensors. Baselines receive the bytes that a learned method spends on its controller. INT8 storage uses a separate symmetric scale for each token/head vector. Both key and value tensors are gathered into compact contiguous storage after eviction. No mask-only saving is counted.

The implementation dequantizes previous entries for eager attention, appends the new chunk, and repacks the retained state. Transient allocations can exceed the resident budget; allocated and reserved CUDA peaks are therefore reported separately. This is a correctness-oriented reference implementation without a fused INT8 attention kernel or a production paging engine.

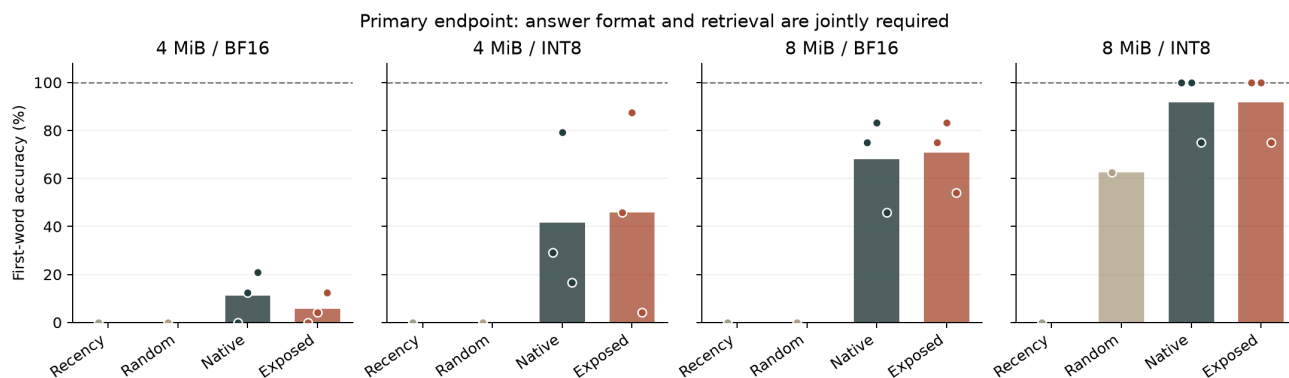
Resident budget	Storage	Baseline slots per head	Learned-policy slots per head
4 MiB	BF16	144	80
4 MiB	INT8	279	155
8 MiB	BF16	288	224
8 MiB	INT8	558	434

The controller consumes about 44% of the smaller budget. Slot counts include the protected window. The reported resident limit applies after compression; processing the next chunk temporarily creates more entries.

Primary outcomes

The primary endpoint requires the normalized first alphabetic word of the generated answer to equal the target color. It measures exact-format task success, including retention of the instruction to answer with one color word. The full-cache model scores 24/24. There are **24 held-out task units**, not 792 independent examples: the 792 rows reuse these tasks across 33 conditions.

Budget / storage	Recency	Random	Native policy, seeds 41 / 42 / 43	Exposed policy, seeds 41 / 42 / 43
4 MiB / BF16	0/24	0/24	0 / 3 / 5 of 24	0 / 1 / 3 of 24
4 MiB / INT8	0/24	0/24	7 / 19 / 4 of 24	11 / 21 / 1 of 24
8 MiB / BF16	0/24	0/24	18 / 20 / 11 of 24	18 / 20 / 13 of 24
8 MiB / INT8	0/24	15/24	24 / 24 / 18 of 24	24 / 24 / 18 of 24



Exposure effects are small relative to the variation across fitted policies. Descriptive paired differences, averaged over the three fixed seeds, are:

Budget / storage	Exposed minus native	Exploratory 95% task-bootstrap interval
4 MiB / BF16	-5.56 percentage points	[-12.50, 0.00]
4 MiB / INT8	+4.17 percentage points	[-2.78, 11.11]
8 MiB / BF16	+2.78 percentage points	[-2.78, 8.33]
8 MiB / INT8	0.00 percentage points	[-4.17, 4.17]

These post-run descriptive intervals resample the 24 task units with seed outcomes kept paired, using 10,000 resamples. They are conditional on the three fitted seeds, omit training-seed population uncertainty, and are not adjusted for four comparisons. They are not confirmatory significance tests. The evidence does not establish a repeatable exposure advantage.

At 8 MiB / INT8, native policies correctly answer 36/36 lookup seed-task pairs and 30/36 correction pairs; exposed policies answer 35/36 and 31/36. Those denominators are repeated tasks over seeds, not independent samples. The weaker seed is material and must not be hidden behind the two perfect seeds.

An endpoint confound found in the audit

The raw baseline answers exposed a problem with interpreting the primary endpoint as factual retrieval. Many recency answers began, “The current access color for archive-...” and reached the 12-token cap before naming any color. Losing the early format instruction can therefore produce a failure without losing the queried fact.

After inspecting the completed primary run, we froze a separate diagnostic. It uses the same 24 prompts and unchanged policies, raises only the answer cap to 64 tokens, and evaluates full cache plus all four compressed methods at 8 MiB / INT8. Learned methods use seed 41, the first registered seed. This is an exploratory follow-up, not a replacement of the primary run.

Method	Original first-word endpoint	First mentioned color correct	Reached 64-token cap
Full cache	24/24	24/24	0/24
Sink + recency	0/24	16/24	0/24
Seeded random	15/24	19/24	0/24
Native policy, seed 41	24/24	24/24	0/24
Exposed policy, seed 41	24/24	24/24	0/24

Recency mentions the correct color for all 12 correction tasks and four of 12 lookups, despite scoring zero under the original first-word rule. This directly refutes the interpretation that its primary zero necessarily means complete factual forgetting. The relaxed metric is also imperfect: it can credit a guess or an early color in an ambiguous response. All 120 diagnostic outputs are preserved. There is no independent human semantic evaluation.

The diagnostic strengthens a measurement lesson, not a state-of-the-art claim. A future benchmark should separate output-format adherence, task semantics, abstention, and truncation, and should protect or repeat the instruction consistently across methods before interpreting a retention advantage.

Resource results

The full cache’s mean measured resident state is 25.33 MiB. Learned retention at 8 MiB / INT8 uses 7.99 MiB including the controller: about **68.5% less resident cache-plus-controller memory**, or 3.17 times smaller. This is a storage result for this workload.

Peak allocated GPU memory is 2.953 GiB for full cache and 2.939 GiB for that learned configuration, a reduction of about **13.8 MiB**, under 0.5% of the full peak. Model weights and temporary activations dominate at these short contexts. Reserved CUDA memory changes from 3.141 to 3.123 GiB. Neither figure is whole-process VRAM usage; the CUDA caching allocator and other applications are separate.

Configuration	Median prefill per task	Median total per task
Full BF16 cache	507 ms	543 ms
Native, 8 MiB / INT8, seed 41	815 ms	878 ms
Exposed, 8 MiB / INT8, seed 41	809 ms	871 ms

These are observed workstation timings, not a controlled serving benchmark. Desktop/browser work occurred during the run, conditions ran sequentially, and answer lengths differ in some settings. Even the two matched perfect-answer learned conditions are slower than full cache here. This implementation demonstrates real packed storage and eviction, **not an inference speedup, an energy saving, or a larger model fitted into 8 GiB.**

Relation to current work

KVP learns per-head retention rankings from generation traces and motivates this pilot’s objective. ForesightKV, LKV, and DistillCache are relevant learned-policy and allocation comparators. HqeKV already studies combined cache quantization and eviction. AgentKV studies phase-sensitive retention and a serving implementation with persistent compressed state. Combining learning, quantization, and eviction is therefore not itself a novelty claim.

This pilot’s contribution is a small reproducible robustness test with controller-inclusive byte accounting and an explicit output-format diagnostic. Competitive claims would require strong implementations such as SnapKV and these contemporary methods, longer-context task suites, more models and seeds, and measurements in an optimized serving engine. Literature coverage here is bounded and does not establish a new research priority.

Limits and next decisive tests

The exposure mechanism does not model repeated self-induced mistakes. Native and exposed training differ only through one fixed preamble eviction and INT8 history; most earlier retained states are merely quantized versions of intact states. Its null result is not evidence against all on-policy or quantization-aware policy training.

The test set is small, synthetic, and in the same generator family as training. There are only three policy seeds, one model, and no independent replication. Controller hyperparameters and training compute are deliberately small. Absolute position features, a fixed head budget, early format instructions, a short generation cap, and teacher-attention utility can all influence the result. Attention utility is a proxy, not a causal label for downstream answer correctness.

Equal bytes intentionally give INT8 more slots. The experiment lacks equal-slot controls and an unpruned INT8 condition, so it does not separately identify quantization error versus increased retention capacity. Its random baseline is a negative control, not a strong competitive method.

The next useful tests are: (1) a fresh instruction-controlled retrieval and correction set with semantic scoring and sufficient answer length; (2) learned policies trained on their own repeatedly compressed histories, with a held-out budget and domain; and (3) actual peak-memory and latency comparisons at longer contexts in a fused or paged implementation. These are proposed follow-ups, not completed results.

Reproducibility and audit trail

The repository preserves the primary configuration, runner, correctness tests, analysis, primary rows, all-condition summary, diagnostic rows, and model/software hashes. No paid GPU or external model API was used for this pilot. Training traces and final controller weights are local artifacts; the runner deterministically regenerates task definitions and records training seeds. CUDA arithmetic was not forced bitwise deterministic.

Correctness checks cover real INT8 storage, reconstruction error, contiguous allocation, byte budgets including controller parameters, absolute positions after eviction, exact BF16 cache parity with stock DynamicCache at identical chunk shapes, and causal isolation from altered future tokens. An initial smoke attempt stopped before task outcomes because an inappropriate full-sequence-versus-chunk BF16 logit tolerance failed; the corrected check compares identical execution shapes. That failed manifest and the implementation log are retained.

The fixed primary run completed 792/792 rows with 33 complete conditions and no duplicate task-condition keys. Every recorded compressed resident peak fits its stated budget. The post-primary diagnostic completed 120/120 rows. One research assistant implemented, checked, and interpreted the study; these checks are not independent scientific review.

Publication record

This edition was selected for Lyrebird on September 22, 2026. The completed primary pilot and separately frozen post-primary diagnostic are reported together. All primary conditions and the failed initial implementation smoke are retained. The diagnostic does not replace the original endpoint.

Research assistance: Codex. Alireza Afshan is the project owner and named author; assistance is disclosed and does not constitute independent review. No journal acceptance or DOI is claimed.

[Versioned source and supporting records](#) · [Download Markdown](#) · [Download BibTeX](#)